Framework di Market Making Robusto via World Models e Trasporto Ottimo

Architettura neurale e procedure di training

Nicolò Fiordaligi
Politecnico di Torino — Ingegneria Matematica MSc

Sommario

Le strategie di market making sono intrinsecamente sviluppate e validate all'interno di simulatori agent-based. Questo introduce una dipendenza strutturale dalla dinamica nominale del simulatore: le policy apprese tendono a sfruttarne le ipotesi specifiche e possono degradare significativamente quando le condizioni operative reali se ne discostano. Il framework propone di affrontare questo problema valutando una strategia non solo nel caso nominale, ma nel caso peggiore in un intorno ragionevole della dinamica prevista, formalizzando questa idea tramite robustezza distribuzionale su palla di Wasserstein.

Il documento è organizzato in cinque sezioni. Le prime tre descrivono i moduli costitutivi del framework: un encoder Transformer del Limit Order Book, un world model neurale autoregressivo che apprende la dinamica nominale nello spazio latente come Gaussian Mixture Model, e un layer di Distributionally Robust Optimization che formula e risolve il problema duale di Wasserstein per costruire il backup di Bellman avversariale one-step. La quarta sezione descrive la pipeline sequenziale di addestramento, che include il training offline del critico e la procedura di stress testing distribuzionale, il cui output principale è la curva $V^{\mathrm{rob}}(\varepsilon)$ al variare del raggio dell'ambiguity set. La quinta sezione descrive l'estensione naturale verso una policy robusta end-to-end tramite Soft Actor-Critic e rollout avversariali multi-step.

1. Modulo A: Encoder del Limit Order Book

Il Modulo A ha l'obiettivo di proiettare lo snapshot discreto del Limit Order Book (LOB) in uno spazio latente continuo e compatto, così da ottenere una rappresentazione più stabile e informativa della microstruttura di mercato. Tale rappresentazione costituisce lo stato su cui operano il world model nominale e, successivamente, il modulo di ottimizzazione robusta.

1.1 Architettura neurale

Sia $o_t \in \mathbb{R}^{L\times F}$ l'osservazione del LOB al tempo t, dove L indica la profondità del book e F il numero di feature per livello, ad esempio prezzi e volumi bid/ask. L'encoder E_θ è implementato come un Transformer Encoder seguito da un layer di proiezione lineare che restituisce uno stato latente $z_t \in \mathbb{R}^d$.

L'architettura dell'encoder è articolata nei seguenti blocchi:

- **Input embedding:** i livelli del LOB vengono proiettati in uno spazio di embedding comune.
- **Self-attention:** N layer di multi-head self-attention modellano le interdipendenze tra top-of-book e livelli profondi.
- **Proiezione latente:** l'output finale viene compresso in un vettore latente z_t di dimensione d.
- **Regolarizzazione geometrica:** spectral normalization e connessioni residue sono introdotte per controllare la sensibilità della mappa e favorire una rappresentazione latente più stabile dal punto di vista geometrico.

In questa formulazione, l'obiettivo non è rivendicare una garanzia formale piena di bi-Lipschitzianità, ma costruire un encoder che preservi in modo ragionevole la struttura del segnale microstrutturale e renda più naturale la successiva ottimizzazione robusta nello spazio latente.

1.2 Procedura di training offline

L'encoder viene addestrato offline sui dati generati dal simulatore nominale tramite uno schema autoencoder. Si introduce un decoder ausiliario D_ψ, utilizzato esclusivamente in fase di training, che ricostruisce lo snapshot del book o sue feature aggregate a partire dal latente $z_t = E_\theta(o_t)$.

La loss di ricostruzione assegna maggior peso ai livelli prossimi al best bid/ask tramite pesi decrescenti w_i. Una possibile formulazione è:

$$\mathcal{L}_{\text{recon}}(\theta, \psi) = \sum_{i=1}^{L} w_i \, \|o_t^{(i)} - D_\psi(E_\theta(o_t))^{(i)}\|_2^2$$

dove $o_t^{(i)}$ denota le feature del livello i-esimo del book. Al termine dell'addestramento, il decoder viene scartato e l'encoder viene congelato, oppure aggiornato con learning rate molto basso nelle fasi successive.

2. Modulo B: World Model nominale autoregressivo

Il Modulo B funge da surrogato differenziabile della dinamica nominale del mercato nello spazio latente. Il suo compito è apprendere, a partire da dati offline generati dal simulatore, la legge di transizione nominale condizionata alla storia recente e alle azioni osservate.

2.1 Variabili in ingresso

Dato l'encoder congelato, ogni osservazione o_t viene trasformata in uno stato latente $z_t = E_\theta(o_t)$. Il world model riceve quindi una finestra causale degli ultimi N stati latenti e delle azioni corrispondenti:

$$X_t = \big[(z_{t-N+1}, a_{t-N+1}), \ldots, (z_t, a_t)\big]$$

Se disponibili, eventuali feature esogene o di contesto, come sentiment analysis, variabili macro o segnali cross-asset, possono essere concatenate **a valle** dell'encoder e utilizzate come covariate addizionali del world model senza modificare la natura microstrutturale del latente z_t.

2.2 Architettura neurale

Il world model M_ϕ è implementato come un Causal Transformer decoder-only. Ciascun token temporale è ottenuto concatenando stato latente e azione, seguiti da una proiezione lineare nella dimensione interna del modello e dall'aggiunta di positional encoding temporale.

La sequenza viene processata da blocchi di masked self-attention, così che il modello possa utilizzare soltanto il passato per costruire una rappresentazione causale della dinamica. L'output associato all'ultimo token produce un vettore di contesto h_t, che sintetizza la storia recente del sistema nello spazio latente.

Da h_t, una testa di tipo Mixture Density Network predice i parametri di una Gaussian Mixture Model a K componenti per la distribuzione del prossimo stato latente. In particolare, il modello approssima la dinamica nominale mediante:

$$P_\phi(z_{t+1} \mid z_{\leq t}, a_{\leq t}) = \sum_{k=1}^{K} \pi_{t,k} \, \mathcal{N}\big(z_{t+1} \mid \mu_{t,k}, \Sigma_{t,k}\big)$$

dove $\pi_{t,k}$ sono i pesi della mixture, $\mu_{t,k} \in \mathbb{R}^d$ sono le medie delle componenti e $\Sigma_{t,k}$ sono matrici di covarianza, tipicamente diagonali per motivi di stabilità numerica e costo computazionale.

Questa scelta consente di modellare transizioni multi-modali e salti locali della microstruttura pur mantenendo uno spazio latente continuo, coerente con la successiva formulazione robusta su palla di Wasserstein.

2.3 Training offline del world model

Il world model viene addestrato offline in modalità teacher forcing, utilizzando sequenze ottenute dal simulatore nominale. Una volta congelato l'encoder, il dataset viene trasformato in tuple latenti del tipo $(z_{t-N+1:t}, a_{t-N+1:t}, r_t, z_{t+1})$.

La funzione obiettivo combina la negative log-likelihood della transizione latente con un termine di regressione sul reward, se si desidera modellare anche la dinamica della ricompensa. Una formulazione possibile è:

$$\mathcal{L}_{\mathrm{WM}}(\phi) = \frac{1}{T} \sum_{t=1}^{T-1} \left[\alpha \, \|\hat{r}_t - r_t\|_2^2 - \log \left(\sum_{k=1}^{K} \pi_{t,k} \, \mathcal{N}(z_{t+1}^{\mathrm{real}} \mid \mu_{t,k}, \Sigma_{t,k}) \right) \right]$$

dove $\hat{r}_t$ è il reward predetto dal modello e z_{t+1}^{real} è lo stato latente successivo osservato nel dataset. Al termine dell'addestramento, il world model viene congelato e trattato come dinamica nominale fissata.

3. Modulo C: Distributionally Robust Optimization su transizioni one-step-ahead

Il Modulo C introduce robustezza rispetto all'incertezza distribuzionale attorno alla dinamica nominale appresa dal world model. In questa formulazione, il simulatore originale non entra più nel loop di ottimizzazione: il nominale è rappresentato dal world model congelato, mentre la robustezza è costruita come un layer matematico sovrapposto alla dinamica latente.

Funzione di valore e critico. Il Modulo C opera su una strategia di market making fissata, di cui si vuole valutare la robustezza distribuzionale. A tale scopo è necessario disporre di una stima della funzione di valore $V : \mathbb{R}^d \to \mathbb{R}$, che assegna a ogni stato latente z il rendimento atteso cumulato della strategia a partire da z. In pratica, V è approssimata da una rete neurale critico $V_\theta(z)$, addestrata offline tramite TD learning su traiettorie latenti generate dal simulatore nominale con la strategia fissata. Una volta addestrato, il critico viene congelato e utilizzato esclusivamente come oggetto di valutazione all'interno del problema duale. La differenziabilità di V_θ rispetto a z è il presupposto che rende possibile la risoluzione dell'inner problem tramite gradient descent .

3.1 Formulazione primale

Dato il contesto causale $h_t = (z_{t-N+1:t}, a_{t-N+1:t})$, il world model congelato fornisce la distribuzione nominale del prossimo stato latente come Gaussian Mixture Model:

$$P_\phi(\cdot \mid h_t) = \sum_{k=1}^{K} \pi_{t,k} \, \mathcal{N}(\cdot \mid \mu_{t,k}, \Sigma_{t,k}) \tag{1}$$

Invece di ottimizzare rispetto alla sola P_ϕ, consideriamo tutte le distribuzioni di transizione Q che non si discostano troppo dalla nominale in senso Wasserstein. Formalmente, l'ambiguity set è:

$$\mathcal{U}_\varepsilon(h_t) = \big\{ Q \in \mathcal{P}(\mathcal{X}) : W_c\big(Q, \, P_\phi(\cdot \mid h_t)\big) \leq \varepsilon \big\} \tag{2}$$

dove il costo di trasporto è $c(x, y) = \|x - y\|_2^2$ tra punti dello spazio latente $x, y \in \mathbb{R}^d$, e W_c è la distanza di Wasserstein indotta da tale costo. Si noti che W_c è definita su *distribuzioni*, mentre c confronta *punti* dello spazio latente: sono oggetti di natura diversa.

Il robust Bellman one-step è allora:

$$V_{\text{rob}}(z_t, a_t) = r(z_t, a_t) + \gamma \inf_{Q \in \mathcal{U}_\varepsilon(h_t)} \mathbb{E}_{z' \sim Q}[V(z')] \tag{3}$$

Il segno inf riflette il fatto che l'avversario, data la policy fissata, sceglie la distribuzione di transizione che minimizza il valore futuro atteso.

3.2 Derivazione formale della formulazione duale

Il problema primale consiste nel ricercare la peggiore distribuzione di transizione Q all'interno dell'intorno di incertezza (ambiguity set) di Wasserstein. Tale problema può essere formulato ricercando il **piano di trasporto ottimale** $\pi(dx, dy) \in \mathcal{P}(\mathcal{X} \times \mathcal{X})$ che minimizza il valore atteso della funzione obiettivo:

$$\inf_{\pi \geq 0} \int_{\mathcal{X} \times \mathcal{X}} V(x) \pi(dx, dy)$$

soggetto ai seguenti vincoli di ammissibilità:

1. **Vincolo marginale su** y: $\int_{\mathcal{X}} \pi(dx, dy) = P_\phi(dy), \quad \forall y \in \mathcal{X}$. Questo vincolo garantisce che la massa probabilistica di partenza sia ancorata alla dinamica nominale e assicura implicitamente la normalizzazione dell'intero piano.
2. **Vincolo di costo Wasserstein**: $\int_{\mathcal{X} \times \mathcal{X}} c(x, y) \pi(dx, dy) \leq \varepsilon$, dove $c(x, y) = \|x - y\|_2^2$ rappresenta lo sforzo di trasporto quadratico tra lo stato nominale e quello perturbato.

Per derivare la forma duale, introduciamo i moltiplicatori di Lagrange associati ai vincoli:

- $\lambda \geq 0$ (moltiplicatore scalare) per il vincolo globale di costo ε.
- $\psi(y)$ (moltiplicatore funzionale) per il vincolo di marginale su y. Poiché esiste un vincolo di uguaglianza per ogni punto dello spazio latente, $\psi(y)$ assume la forma di un potenziale di Kantorovich.

Il problema duale associato ha la forma:

$$\sup_{\lambda \geq 0,\, \psi} \inf_{\pi \geq 0} \mathcal{L}(\pi, \lambda, \psi)$$

dove la Lagrangiana è definita come:

$$\mathcal{L}(\pi, \lambda, \psi) = -\lambda\varepsilon + \int \psi(y)\, P_\phi(dy) + \int_{\mathcal{X} \times \mathcal{X}} \big[V(x) + \lambda c(x, y) - \psi(y)\big]\, \pi(dx, dy)$$

L'infimum rispetto a $\pi \geq 0$ è finito se e solo se il termine tra parentesi quadre è non-negativo per ogni coppia (x, y), ovvero:

$$\psi(y) \leq V(x) + \lambda \|x - y\|_2^2 \quad \forall x \in \mathcal{X}$$

Poiché la funzione duale contiene il termine $\int \psi(y)\, P_\phi(dy)$, che si vuole massimizzare rispetto a ψ, la scelta ottimale è rendere $\psi(y)$ il più grande possibile compatibilmente con questo vincolo, ovvero saturarlo puntualmente:

$$\psi(y) = \inf_{x \in \mathcal{X}} \big\{V(x) + \lambda \|x - y\|_2^2\big\}$$

Sostituendo tale espressione nella Lagrangiana, i termini dipendenti dal piano di trasporto π si annullano. L'integrale $\int \psi(y) P_\phi(dy)$ si trasforma nel valore atteso rispetto alla distribuzione nominale, portando alla **formulazione duale esatta**:

$$\boxed{\inf_{Q \in \mathcal{U}_\varepsilon(h_t)} \mathbb{E}_Q[V(z')] = \sup_{\lambda \geq 0} \left\{ -\lambda\varepsilon + \mathbb{E}_{y \sim P_\phi(\cdot|h_t)} \left[\inf_{x \in \mathcal{X}} \big(V(x) + \lambda \|x - y\|_2^2 \big) \right] \right\}} \tag{4}$$

Questa riformulazione analitica giustifica la scomposizione del Modulo C in tre livelli di ottimizzazione: l'aggiornamento esterno di λ, l'aspettazione tramite campionamento di y, e la ricerca interna dello stato avversariale x^* tramite discesa del gradiente.

3.3 Strategia di risoluzione approssimata

Il problema duale è un'ottimizzazione annidata a tre livelli. La procedura di risoluzione si articola nelle seguenti fasi sequenziali: (i) campionamento di m punti nominali y_j dalla GMM del world model; (ii) risoluzione di m problemi di ottimizzazione indipendenti in x tramite gradient descent iterativo, uno per ciascun y_j; (iii) stima Monte Carlo della funzione duale e aggiornamento del moltiplicatore λ via projected ascent, con gradiente disponibile in forma chiusa per il teorema dell'inviluppo.

Fase 1 — Campionamento dei punti nominali (middle): dato il contesto h_t, il world model produce i parametri della GMM $\{\pi_{t,k}, \mu_{t,k}, \Sigma_{t,k}\}_{k=1}^K$. Si ottengono m punti nominali con una delle due strategie:

- *Campionamento Monte Carlo:* $y_j \sim P_\phi(\cdot \mid h_t)$, per $j = 1, \ldots, m$;
- *Centroidi pesati:* $y_k = \mu_{t,k}$ con peso $\pi_{t,k}$, per $k = 1, \ldots, K$; più stabile quando le varianze della GMM sono piccole, poiché elimina la varianza dello stimatore Monte Carlo.

Ogni y_j rappresenta un possibile next state latente secondo la dinamica nominale del world model.

Fase 2 — Ottimizzazione avversariale per ogni y_j (inner): per ciascun punto nominale y_j, si cerca il next state avversariale $x^*(y_j, \lambda)$ che massimizza il danno alla policy rimanendo vicino a y_j:

$$x^*(y_j, \lambda) = \arg\min_{x \in \mathbb{R}^d} \big(V(x) + \lambda \|x - y_j\|_2^2 \big)$$

Qui x non è un campione stocastico ma una **variabile di ottimizzazione continua**: il termine $V(x)$ spinge l'avversario verso stati che degradano il valore della policy, mentre $\lambda\|x - y_j\|_2^2$ penalizza l'allontanamento dal nominale. Il problema viene approssimato con M passi di gradient descent, inizializzando $x^{(0)} = y_j$:

$$x^{(m+1)} = x^{(m)} - \eta \, \nabla_x \big(V(x^{(m)}) + \lambda \|x^{(m)} - y_j\|_2^2 \big), \quad m = 0, \ldots, M-1 \tag{5}$$

Il gradiente ha forma esplicita:

$$\nabla_x \big(V(x) + \lambda \|x - y_j\|_2^2 \big) = \nabla_x V(x) + 2\lambda(x - y_j)$$

dove $\nabla_x V(x)$ è calcolato via backpropagation attraverso la value network, e $2\lambda(x - y_j)$ è il termine analitico della penalità. Al termine delle M iterazioni si ottiene $x_j^* \approx x^*(y_j, \lambda)$. Questo procedimento viene eseguito indipendentemente per ciascuno degli m punti nominali.

Nota. Poiché V_θ è una rete neurale non convessa, il gradient descent converge in generale a un minimo locale di $V(x) + \lambda\|x - y_j\|_2^2$, non necessariamente al minimo globale. Il gradiente di λ e la condizione di complementary slackness descritta nella Fase 3 sono pertanto da intendersi come

condizioni asintotiche al crescere di M, coerentemente con quanto documentato in letteratura per problemi WDRO con funzioni obiettivo non convesse.

Fase 3 — Aggiornamento di λ (outer): una volta ottenuti i m stati avversariali $\{x_j^*\}$, si stima la funzione duale:

$$\hat{g}(\lambda) = -\lambda\varepsilon + \frac{1}{m}\sum_{j=1}^{m}\big(V(x_j^*) + \lambda\|x_j^* - y_j\|_2^2\big) \tag{6}$$

Per il calcolo del gradiente si applica il **teorema dell'inviluppo**: poiché x_j^* è un minimizzatore del problema interno, le condizioni di primo ordine in x annullano la derivata totale rispetto a λ, lasciando solo la derivata parziale esplicita. Il gradiente si riduce alla forma chiusa:

$$\nabla_\lambda \hat{g}(\lambda) = -\varepsilon + \frac{1}{m}\sum_{j=1}^{m}\|x_j^* - y_j\|_2^2 \tag{7}$$

Questo risultato ha un'interpretazione precisa: alla soluzione ottima λ^*, e sotto l'ipotesi che l'inner problem sia risolto esattamente, si ha $\frac{1}{m}\sum_j \|x_j^* - y_j\|_2^2 = \varepsilon$, ovvero il vincolo Wasserstein è attivo all'ottimo (*complementary slackness*). Il moltiplicatore viene aggiornato con projected ascent:

$$\lambda \leftarrow \max(0,\ \lambda + \alpha \cdot \nabla_\lambda \hat{g}(\lambda)) \tag{8}$$

Il costo computazionale di questo update è trascurabile: il gradiente è disponibile in forma chiusa una volta completata la Fase 2, senza backpropagation aggiuntiva.

3.4 Backup robusto e metrica di stress testing

Una volta completate le tre fasi, si dispone di m stati avversariali $\{x_j^*\}_{j=1}^m$. Il target robusto del Bellman one-step viene costruito scegliendo lo stato peggiore tra i m:

$$x_{t+1}^* = \arg\min_{j\in\{1,\dots,m\}} V_\theta(x_j^*) \tag{9}$$

$$y_t^{\text{rob}} = r(z_t, a_t) + \gamma\, V_\theta(x_{t+1}^*) \tag{10}$$

In alternativa, si può usare la media $\frac{1}{m}\sum_j V_\theta(x_j^*)$ come target, più stabile ma meno conservativa.

Il world model rimane congelato e rappresenta esclusivamente la dinamica nominale, mantenendo netta la separazione tra apprendimento della dinamica nominale e layer robusto sovrapposto.

Natura one-step della perturbazione. È importante sottolineare che x_{t+1}^* non viene reimmesso nella context window del Causal Transformer: il contesto h_t utilizzato ad ogni passo è costruito interamente dalla traiettoria nominale osservata $\{z_s\}_{s\le t}$. Le perturbazioni ai diversi passi temporali sono quindi *indipendenti* tra loro: ad ogni t, il Modulo C chiede localmente qual è il next state avversariale peggiore nell'intorno ε della distribuzione nominale, senza propagare la perturbazione ai passi successivi.

Questo implica che $V^{\text{rob}}(\varepsilon)$ è un *lower bound conservativo* sul valore nominale, ottenuto perturbando ogni transizione in modo ottimale ma non coerente temporalmente.

Metrica di stress testing. L'output principale del framework, per una strategia fissata, è la curva di valore robusto:

$$V^{\text{rob}}(\varepsilon) = \mathbb{E}_\tau\left[\sum_t \gamma^t\, y_t^{\text{rob}}(\varepsilon)\right] \tag{11}$$

calcolata applicando il Modulo C su un insieme di traiettorie nominali τ al variare del raggio ε dell'ambiguity set. La curva $V^{\text{rob}}(\varepsilon)$ quantifica la degradazione del rendimento atteso della

strategia sotto perturbazioni avversariali di intensità crescente: per $\varepsilon = 0$ coincide con il valore nominale, mentre al crescere di ε cattura scenari di mercato progressivamente più distanti dalla dinamica appresa dal world model.

Questa metrica costituisce lo strumento di valutazione principale del framework nella sua forma di stress testing: permette di confrontare strategie diverse sotto lo stesso livello di perturbazione, e di identificare la soglia ε^* oltre la quale il rendimento atteso della strategia degrada significativamente.

4. Pipeline complessiva

L'intero framework si articola in fasi sequenziali che separano esplicitamente l'apprendimento della dinamica nominale, la stima del valore della strategia, e la valutazione robusta.

- **Fase 1 — Training offline dell'encoder:** l'encoder E_θ viene addestrato su dati generati dal simulatore nominale tramite uno schema autoencoder con decoder ausiliario D_ψ e loss di ricostruzione pesata sui livelli del LOB. Al termine, D_ψ viene scartato e E_θ viene congelato.
- **Fase 2 — Costruzione del dataset latente:** con l'encoder congelato, si eseguono rollout del simulatore nominale con la strategia da valutare e si costruisce un dataset latente di tuple $(z_{t-N+1:t},\, a_{t-N+1:t},\, r_t,\, z_{t+1})$.
- **Fase 3 — Training offline del world model:** il world model M_ϕ viene addestrato sul dataset latente in modalità teacher forcing, minimizzando la NLL della GMM sulla transizione e, opzionalmente, l'MSE sul reward. Al termine, M_ϕ viene congelato e trattato come dinamica nominale fissata.
- **Fase 4 — Training offline del critico:** il critico V_θ viene addestrato sul dataset latente tramite TD learning, stimando il valore atteso cumulato della strategia fissata a partire da ogni stato latente. Al termine, V_θ viene congelato e utilizzato esclusivamente come oggetto di valutazione nel Modulo C.
- **Fase 5 — Stress testing distribuzionale:** il Modulo C viene applicato su un insieme di traiettorie nominali generate dalla strategia fissata, al variare del raggio ε dell'ambiguity set. Per ogni traiettoria e ogni passo temporale, il duale Wasserstein viene risolto tramite la procedura descritta nella Sezione 3, producendo il backup robusto one-step. L'output è la curva $V^{\text{rob}}(\varepsilon)$, che quantifica la degradazione del rendimento atteso della strategia sotto perturbazioni avversariali di intensità crescente.

Nota sul ruolo del simulatore: il simulatore nominale (es. ABIDES) viene utilizzato esclusivamente nelle Fasi 1–2 per la generazione offline dei dati. Non entra mai nel loop di ottimizzazione del Modulo C. Il vantaggio computazionale principale del framework risiede nella differenziabilità del world model: essa rende possibile la formulazione e la risoluzione del problema DRO tramite ottimizzazione basata su gradienti, operazione impossibile con il simulatore diretto.

5. Estensione naturale: policy robusta end-to-end

Il framework descritto nelle Sezioni 1–4 si concentra sulla valutazione robusta di una strategia fissata tramite stress testing distribuzionale. Un'estensione diretta consiste nell'utilizzare il backup di Bellman avversariale prodotto dal Modulo C non come metrica di valutazione, ma come target per addestrare una policy di market making robusta end-to-end.

In questa configurazione, la policy π_ω è una rete neurale che opera sul vettore di contesto $h_t \in \mathbb{R}^{d_h}$ prodotto dal Causal Transformer congelato. L'algoritmo naturale è **Soft Actor-Critic (SAC)**, scelto per tre ragioni: supporto nativo agli spazi d'azione continui richiesti dal market making,

efficienza campionaria tramite replay buffer latente, e regolarizzazione entropica che previene il collasso della policy in presenza di un world model imperfetto.

Integrazione con il backup robusto. Il target robusto y_t^{rob} prodotto dal Modulo C sostituisce direttamente il target nominale nel Bellman backup di SAC. Il next state avversariale x_{t+1}^* viene passato attraverso il Causal Transformer congelato per aggiornare il contesto:

$$h_{t+1} = \text{CausalTransformer}\big(z_{t-N+2:t},\, x_{t+1}^*,\, a_{t-N+2:t+1}\big) \tag{12}$$

Il target robusto diventa:

$$y_t^{\text{rob}} = r_t + \gamma \left(\min_i \bar{Q}_{\phi_i}(h_{t+1},\, a') \;-\; \alpha \log \pi_\omega(a' \mid h_{t+1}) \right), \quad a' \sim \pi_\omega(\cdot \mid h_{t+1}) \tag{13}$$

È essenziale distinguere i ruoli dei due oggetti: x_{t+1}^* determina il contesto h_{t+1} su cui actor e critic sono valutati, rendendo il critic conservativo rispetto al caso peggiore; il termine entropico $-\alpha \log \pi_\omega(a' \mid h_{t+1})$ mantiene invece la struttura esplorativa del SAC standard, invariata rispetto al caso nominale. Il training avviene interamente nello spazio latente tramite replay buffer latente, senza interazione diretta con il simulatore ad ogni step.

Verso rollout avversariali multi-step. L'ulteriore estensione naturale consiste nel propagare x_{t+1}^* nella context window del Transformer, generando traiettorie avversariali coerenti temporalmente. In questa configurazione il kernel avversario $Q^*(\cdot \mid h_t)$ viene costruito esplicitamente, superando il limite dell'indipendenza delle perturbazioni one-step e producendo scenari di stress globalmente coerenti lungo l'intera traiettoria.